

JavaScript ES6+ for Hotel Data Structures

Objectives

By the end of this session, you should be able to:

- Apply modern JavaScript (ES6+) features to hotel data structures
 - Use variables, functions, and data types for hotel booking logic
 - Implement array methods for hotel management
 - Use template literals for hotel messaging
 - Apply destructuring for hotel data extraction
 - Implement spread/rest operators for hotel operations
 - Create hotel data structures using objects and arrays
 - Use classes for hotel data modeling
 - Handle asynchronous operations with Promises
 - Work with modern JavaScript features (optional chaining, nullish coalescing)
-

Hotel Booking Project Context

Throughout this course, you'll build a complete **Hotel Room Booking Service**. Today, we'll start with the fundamental JavaScript skills needed to handle hotel data.

Key Hotel Data Structures We'll Work With:

- Hotels (name, location, rating, amenities **/ə'menətɪz/**)
 - Rooms (type, price, capacity, availability)
 - Bookings (dates, guest info, room assignment)
 - Guests (name, contact, preferences)
-

Setup

1. Run JavaScript in a Web Browser

Steps:

1. Open any browser (Chrome, Edge, Firefox, Safari).
2. Open Developer Tools:
 - Windows/Linux: Ctrl + Shift + I
 - Mac: Cmd + Option + I
1. Go to the Console tab.
2. Type:

```
console.log("Hello World");
```

1. Press Enter to run.
-

2. Run JavaScript in an HTML File

Steps:

1. Open a text editor (VS Code, Notepad, etc.).
2. Create a file called `index.html`.
3. Add:

```
<html>
  <body>
    <script>
      console.log("Hello from JS");
      alert("Hello!");
    </script>
  </body>
</html>
```

1. Save the file.

2. Open it in a browser.

3. Run JavaScript using Node.js

Steps:

1. Install Node.js: <https://nodejs.org>
2. Check installation:

```
node -v
```

1782275253268

1. Create a file **app.js**.
2. Add:

```
console.log("Running JavaScript in Node.js");
```

1. Run:

```
node app.js
```

Summary

- Browser Console → quick testing
 - HTML file → frontend development
 - Node.js → backend development
-

```
# Check Node.js version  
node --version
```

```
# Create project file
touch hotel-data.js
```

hotel-data.js

```
console.log("Welcome to Hotel Booking System!");
```

Run your code:

```
node hotel-data.js
```

JavaScript Cheatsheet: <https://quickref.me/javascript.html>

Activity 1: Variables for Hotel Data

let, const, and var for Hotel Data

Understanding Variables:

- **let**: Block-scoped. Can be updated but not redeclared in the same scope.
- **const**: Block-scoped. Must be assigned at declaration and cannot be updated or redeclared.
- **var**: Function-scoped. Can be updated and redeclared. *Avoid using* due to confusing scoping and hoisting behavior.

Key difference: var vs let

The important differences:

1. Scope

var → function-scoped

let → block-scoped { }

```
if (true) {  
  var x = 10;  
}  
console.log(x); // works (still accessible)  
  
if (true) {  
  let y = 10;  
}  
console.log(y); // error (not accessible)
```

2. Hoisting

var is hoisted (declared at top, value = undefined)

let is hoisted but not initialized (temporal dead zone)

```
console.log(a); // undefined  
var a = 5;  
  
console.log(b); // error  
let b = 5;
```

3. Re-declaration

```
var a = 1; var a = 2; // allowed  
  
let b = 1; let b = 2; // error
```

Hotel Example - Step by Step:

```
// GOOD: Using const for hotel constants  
const HOTEL_NAME = "Grand Palace Hotel";  
const MAX_ROOMS = 100;  
const BASE_PRICE = 150;  
  
// GOOD: Using let for values that change  
let availableRooms = 45;  
let bookedRooms = 55;  
let currentDiscount = 0.1;  
  
// BAD: Using var (avoid this)  
var oldStyleVariable = "don't use this";
```

```
// Block scope demonstration
if (availableRooms > 0) {
  let discount = 0.1; // 10% discount
  const specialOffer = "Summer Sale";
  console.log("Inside block:", discount); // Works
}

// console.log(discount); // Error: not defined outside block
console.log("Outside block:", availableRooms); // Works
```

Your Turn:

```
// TODO: Create your own hotel variables
// 1. Create a const for your hotel name
// 2. Create a const for maximum room capacity
// 3. Create a let for current available rooms
// 4. Create a let for current price per night
// 5. Try to update the let variables
// 6. Try to reassign the const variables (what happens?)

// Your code here:

// Your output here:
```

Always use **const** by default, only use **let** when you know the value will change.
Never use **var** in modern JavaScript.

Data Types

There are two types of data in JavaScript:

- Primitive Data Types:
 - String
 - Number
 - Boolean
 - Null
 - Undefined
 - Symbol
 - BigInt
- Objects > Beside Primitive, everything else is an Object:
 - Array

- Date
- RegExp
- Function

```
// Primitive Types
let hotelName = "Sunshine Hotel"; // String
let roomPrice = 150; // Number
let available = true; // Boolean
let manager = null; // Null
let guestCount; // Undefined
let roomId = Symbol("room"); // Symbol
let largeNumber = 12345678901234567890n; // BigInt

// Object Types
let rooms = ["101", "102", "103"]; // Array
let bookingDate = new Date(); // Date
let pattern = /hotel/i; // RegExp

function welcome() { // Function
    return "Welcome!";
}

let hotel = { // Object
    name: "Sunshine Hotel",
    rooms: 50
};
```

Operators and Features

1. Arithmetic Operators

+, -, *, /, %

```
let result = 10 + 5; // 15
let remainder = 10 % 3; // 1
```

2. Assignment Operators

=, +=, -=, *=, /=

```
let x = 10;
x += 5; // 15
```

3. Comparison Operators

==, ===, !=, !==, <, >, <=, >=

```
5 === "5"; // false
5 > 3; // true
```

4. Logical Operators

&&, ||, !

```
true && false; // false
true || false; // true
!true; // false
```

5. Bitwise Operators

&, |, ^, ~, <<, >>, >>>

```
5 & 1; // 1
5 | 1; // 5
```

6. Unary Operators

++, --

```
let count = 1;
count++; // 2
count--; // 1
```

7. Ternary Operator

```
let age = 18;
let status = age >= 18 ? "Adult" : "Minor";
```

8. Spread Operator (...)


```
let nums = [1, 2, 3];
let newNums = [...nums, 4];
```

9. Rest Operator (...)

```
function sum(...numbers) {
  return numbers.length;
}
```

10. Object Destructuring

```
const hotel = { name: "Sunshine", rooms: 50 };
const { name, rooms } = hotel;
```

11. Array Destructuring

```
const colors = ["red", "blue"];
const [first, second] = colors;
```

Conditional Statements

12. If Statement

```
if (roomPrice > 100) {
  console.log("Expensive");
}
```

13. Switch Statement

```
let day = 1;

switch(day) {
  case 1:
    console.log("Monday");
    break;
}
```

Loops

14. For Loop

```
for(let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

15. While Loop

```
let i = 0;  
  
while(i < 3) {  
  console.log(i);  
  i++;  
}
```

Exception Handling

16. try...catch

```
try {  
  throw new Error("Oops");  
} catch(error) {  
  console.log(error.message);  
}
```

Object-Oriented Programming

17. Classes

```
class Hotel {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
const h = new Hotel("Sunshine");
```

Modern JavaScript Features

18. Modules

```
// export.js
export const name = "Sunshine";

// import.js
import { name } from "./export.js";
```

19. Iterators

```
const arr = [1, 2];
const iterator = arr[Symbol.iterator]();

console.log(iterator.next());
```

20. Generators

```
function* numbers() {
  yield 1;
  yield 2;
}

const gen = numbers();

console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
```

The `*` after the `function` keyword tells JavaScript that this function is a generator.

21. Promises

```
const isRoomAvailable = false;

const booking = new Promise((resolve, reject) => {
  if (isRoomAvailable) {
    resolve("Room booked successfully!");
  } else {
    reject("Sorry, no rooms available.");
  }
});

booking
  .then(message => console.log(message))
  .catch(error => console.log(error));
```

22. Async/Await

```
async function getData() {
  return "Data"; // returns a promise by default
}

const result = await getData();
```

23. Callbacks

```
function greet(callback) {
  callback();
}

greet(() => console.log("Hello"));
```

24. Decorators (Experimental feature)

```
@logged
class User {}
```

25. Enums (Not native JavaScript; available in TypeScript)

```
enum Status {
  Active,
  Inactive
}
```

Collection Types

26. Map

```
const users = new Map();
users.set("name", "Hyden");
users.set("age", 21);

console.log(users.get("name")); // Hyden
console.log(users.get("age")); // 21

console.log(users.has("name")); // true
console.log(users.has("email")); // false

userMap.delete("age");
```

27. Set

```
const numbers = new Set([1, 2, 2, 3]);

numbers.add(4);

console.log(numbers.has(1)); // true
console.log(numbers.has(5)); // false

numbers.delete(1);

console.log(numbers);

///
const arr = [1, 2, 2, 3, 3, 4];

const unique = new Set(arr);

console.log([...unique]);
// [1, 2, 3, 4]
```

28. WeakMap

A **WeakMap** is a special type of **Map** where **keys must be objects**, and those keys are held **weakly**.

That means:

If nothing else is referencing the key object, it can be **garbage collected automatically**.

Rules of WeakMap

Keys must be objects only

```
const wm = new WeakMap();  
  
const obj = { id: 1 };  
  
wm.set(obj, "user data");
```

Primitive keys NOT allowed

```
// wm.set("name", "Oumar"); Error  
// wm.set(123, "value"); Error
```

set / get / has / delete

```
const user = { name: "Cliff" };  
  
const wm = new WeakMap();  
  
wm.set(user, "Premium User");  
  
console.log(wm.get(user)); // Premium User  
console.log(wm.has(user)); // true  
  
wm.delete(user);  
console.log(wm.has(user)); // false
```

Key Feature: Garbage Collection

```
let user = { name: "Alex" };

const wm = new WeakMap();
wm.set(user, "Session Data");

// If we remove reference:
user = null;

// Now the object is eligible for garbage collection
// and WeakMap entry disappears automatically
```

👉 You cannot control or inspect this cleanup - it happens automatically.

Special Data Types

29. Symbol

A **Symbol** is a **unique and immutable primitive value**.

Even if two symbols have the same description, they are still different.

Create a Symbol

```
const id = Symbol("id");
```

Uniqueness

```
const a = Symbol("key");
const b = Symbol("key");

console.log(a === b); // false
```

👉 Same description, but still completely different values.

Use case: object property keys

Symbols are often used to avoid name collisions.

```
const user = {
  name: "Ken"
};
```

```
const id = Symbol("id");

user[id] = 101;

console.log(user[id]); // 101
```

Why Symbol is useful

- Prevents accidental overwriting of object properties
- Used for **hidden/internal properties**
- Used in JavaScript internals (like `Symbol.iterator`)

Example: hidden property

```
const secret = Symbol("secret");

const obj = {
  name: "Bob",
  [secret]: "hidden value"
};

console.log(obj[secret]); // hidden value
console.log(Object.keys(obj)); // ["name"] → symbol is hidden
```

Key idea

Symbols create **non-colliding, hidden keys** in objects.

30. Typed Arrays

Typed Arrays are used for **binary data and high-performance operations**.

They store **raw binary data in fixed formats**, unlike normal arrays.

Example: Int32Array

```
const numbers = new Int32Array([1, 2, 3]);
```



```
console.log(numbers);  
// Int32Array(3) [1, 2, 3]
```

What does Int32Array mean?

- **Int32** → 32-bit signed integer
- Fixed memory size per element
- Much faster for numeric operations

Common Typed Arrays

Type	Description
Int8Array	8-bit integers
Uint8Array	8-bit unsigned integers
Int16Array	16-bit integers
Int32Array	32-bit integers
Float32Array	32-bit floating numbers
Float64Array	64-bit floating numbers

Example: Float array

```
const floats = new Float32Array([1.5, 2.5, 3.5]);  
console.log(floats);
```

Why Typed Arrays exist

They are used for:

- 🧠 WebAssembly
- 🎮 Game development
- 🎧 Audio processing
- 🖼️ Image/video manipulation

-  High-performance computations
-

Difference vs normal Array

Feature	Array	Typed Array
Type flexibility	Yes	No (fixed type)
Performance	Slower	Faster
Memory usage	dynamic	efficient
Methods	full JS methods	limited

Example: binary buffer usage

```
const buffer = new ArrayBuffer(8);
const view = new Int32Array(buffer);

view[0] = 100;
view[1] = 200;

console.log(view); // Int32Array(2) [100, 200]
```

Summary

Most important topics to learn first:

- Variables (let, const)
- Data Types
- Operators
- If / Switch
- Loops
- Functions
- Arrays and Objects
- Classes
- Promises & Async/Await
- Modules

Topics like Decorators, Iterators, Generators, WeakMaps, Symbols, and Typed Arrays are more advanced and can be learned later.

Operators

```
false || true;    // true
10 > 5 || 10 > 20; // true
false && true;     // false
10 > 100 || 10 > 20; // false
```

Logical Operators

```
true && true; // true
true && false; // false
true || false; // true
false || false; // false
!true; // false
!false; // true
```

Comparison Operators

```
10 > 5; // true
10 < 5; // false
10 >= 5; // true
10 <= 5; // false
10 == 5; // false
10 != 5; // true
10 === 5; // false
10 !== 5; // true
"10" == 10; // true
"10" === 10; // false
"10" !== 10; // true
"10" > 5; // true
"10" < 5; // false
"10" >= 5; // true
"10" <= 5; // false
"10" > "5"; // true
"10" < "5"; // false
"10" >= "5"; // true
"10" <= "5"; // false
```

Note: In JavaScript, the == operator performs type coercion, while === does not.

Type coercion: JavaScript converts one type to another to make operations easier. For example, when you compare a string with a number, JavaScript converts the string to a number before comparing.

Your Turn:

```
// TODO: Create your own comparison operators
// Your code here:
console.log(10 == "10");
console.log(10 === "10");
console.log(10 > "5");
```

```
let lateToWork = true;
let oppositeValue = !lateToWork;

console.log(oppositeValue);
```

Nullish coalescing operator ?? The nullish coalescing operator (??) is a logical operator that returns its right-hand side operand when its left-hand side operand is **null** or **undefined**, and otherwise returns its left-hand side operand.

```
null ?? 'I win';           // 'I win'
undefined ?? 'Me too';     // 'Me too'

false ?? 'I lose'         // false
0 ?? 'I lose again'       // 0
'' ?? 'Damn it'           // ''
```

Activity 2: Functions for Hotel Operations

Functions for Hotel Booking

Understanding Functions: Functions are reusable blocks of code that perform specific tasks. In hotel booking, functions help us calculate prices, check availability, and process bookings.

Hotel Booking Functions - Step by Step:

1. Traditional Named Function:

```
function calculateRoomPrice(basePrice, nights, discount) {
  const total = basePrice * nights;
  return total - (total * discount);
}
```

```
}

// Usage
const totalPrice = calculateRoomPrice(100, 3, 0.1); // $270
console.log(`Total price: ${totalPrice}`);
```

2. Arrow Function (Modern & Concise):

```
const calculateRoomPrice = (basePrice, nights, discount = 0) => {
  const total = basePrice * nights;
  return total - (total * discount);
};

// Short form for simple functions
const formatPrice = (price) => `${price.toFixed(2)}`;

console.log(calculateRoomPrice(100, 3, 0.1));
console.log(formatPrice(270)); // "$270.00"
```

3. Function with Multiple Parameters:

```
const calculateTotalCost = (basePrice, nights, guests, taxRate = 0.08) => {
  const subtotal = basePrice * nights * guests;
  const tax = subtotal * taxRate;
  return subtotal + tax;
};

console.log(calculateTotalCost(100, 3, 2)); // $648
```

Your Turn

```
// TODO: Create hotel booking functions
// 1. Create a function to calculate total guests (adults + children)
// 2. Create a function to check if room is available (capacity >= guests)
// 3. Create a function to calculate nights between two dates
// 4. Create a function to apply discount based on nights (7+ nights = 10% off)

// Hint: newDate(date) -> converts a date string and returns a `Date object`
//       Subtracting two Date objects returns the difference in
//       `milliseconds`.
// Your code here:

// Test your functions
// console.log(calculateTotalGuests(2, 3)); // Should return 5
// console.log(isRoomAvailable(4, 3)); // Should return true
// console.log(calculateNights("2026-06-01", "2026-06-08")); // Should
```

```
return 7
// console.log(applyDiscount(1000, 7)); // Should return 900
```

Use default parameter values to make functions more flexible. Arrow functions are preferred for callbacks and simple operations.

Activity 3: Template Literals for Hotel Messages

Template Literals for Hotel Messaging

Understanding Template Literals: Template literals use backticks (``) instead of quotes and allow you to embed expressions directly in strings using `${}`. They also support multi-line strings.

Hotel Booking Examples - Step by Step:

```
const guestName = "Le";
const hotelName = "Grand Palace Hotel";
const roomNumber = 305;
const checkInDate = "2026-06-15";
const totalPrice = 450;

// OLD WAY: String concatenation (messy and hard to read)
const message = "Dear " + guestName + ", your room " + roomNumber + " at " +
hotelName + " is ready.";

// NEW WAY: Template literals (clean and readable)
const modernMessage = `Dear ${guestName}, your room ${roomNumber} at
${hotelName} is ready for check-in on ${checkInDate}`;

console.log(modernMessage);

// Multi-line template literal
const bookingConfirmation = `
Dear ${guestName},

Thank you for booking at ${hotelName}!

Booking Details:
- Room: ${roomNumber}
- Check-in: ${checkInDate}
- Total: $${totalPrice}
```

```
We look forward to your stay!
`;

console.log(bookingConfirmation);

// Template literals with expressions
const nights = 3;
const perNight = 150;
const summary = `Total: ${nights * perNight} for ${nights} nights`;
console.log(summary);
```

Your Turn

```
// TODO: Create template literals for hotel messaging
// 1. Create a booking confirmation message with guest name and room number
// 2. Create a multi-line welcome message with hotel amenities
// 3. Create an invoice summary with calculations
// 4. Create an error message for invalid booking dates

// Your code here:

// Your output here:

// Test your codes
// console.log(bookingConfirmation("John Smith", 205));
// console.log(welcomeMessage("Sunshine Hotel"));
// console.log(invoiceSummary("Le", 3, 120));
// console.log(invalidDateError("2026-06-10", "2026-06-08"));
```

Sample Output

Booking Confirmed!

Guest: Le

Room Number: 205

Welcome to Sunshine Hotel!

Amenities:

- Free Wi-Fi
- Swimming Pool
- Fitness Center
- Complimentary Breakfast

Enjoy your stay!

Invoice Summary

Guest: John Smith

Nights: 3

Price per Night: \$120

Total Amount: \$360

Error: Invalid booking dates.

Check-in Date: 2026-06-10

Check-out Date: 2026-06-08

Check-out date must be after the **check-in** date.

Use template literals for all string operations. They're more readable, support multi-line strings, and handle expressions elegantly.

Practice: Create template literals for:

- Booking confirmation email
- Room availability message
- Guest welcome message
- Invoice summary

Activity 4: Destructuring for Hotel Data

Destructuring for Hotel Data

Understanding Destructuring: Destructuring allows you to extract values from arrays or properties from objects into distinct variables. It makes your code cleaner and more readable.

Hotel Booking Examples - Step by Step:

```
// Object destructuring
const hotel = {
  id: 1,
  name: "Grand Palace Hotel",
  location: "Foshan",
  rating: 4.5,
  amenities: ["WiFi", "Pool", "Spa"]
};

// OLD WAY: Accessing properties individually
const hotelName = hotel.name;
const hotelLocation = hotel.location;
const hotelRating = hotel.rating;

// NEW WAY: Destructuring (clean and concise)
const { name, location, rating } = hotel;
console.log(`${name} in ${location} has ${rating} stars`);

// Destructuring with renaming
const { name: hotelName, location: city } = hotel;
```



```

console.log(`Hotel: ${hotelName}, City: ${city}`);

// Destructuring with default values
const { id, description = "No description available" } = hotel;
console.log(description); // "No description available"

// Array destructuring
const roomTypes = ["Single", "Double", "Suite", "Penthouse"];
const [firstType, secondType, thirdType] = roomTypes;
console.log(`Basic rooms: ${firstType}, ${secondType}, ${thirdType}`);

// Skip elements in array destructuring
const [first, , third] = roomTypes;
console.log(`First: ${first}, Third: ${third}`);

// Destructuring function parameters
function displayBooking({ guestName, roomNumber, checkIn }) {
  console.log(`Booking: ${guestName} - Room ${roomNumber} - ${checkIn}`);
}

const booking = {
  guestName: "Le",
  roomNumber: 305,
  checkIn: "2026-06-15"
};

displayBooking(booking);

```

Your Turn

```

// TODO: Practice destructuring
// 1. Destructure the guest object to extract firstName, lastName, and email
// 2. Destructure the amenities array to get the first two amenities
// 3. Create a function that accepts a destructured hotel object
// 4. Use default values when destructuring

const guest = {
  firstName: "Jane",
  lastName: "Smith",
  email: "jane@example.com",
  phone: "+1234567890",
  preferences: ["non-smoking", "high floor"]
};

// Your code here:

```

Destructuring is especially useful when working with API responses and complex data structures. It makes your code more readable and reduces repetition.

Activity 5: Spread and Rest Operators

Spread and Rest for Hotel Operations

Understanding Spread and Rest:

- **Spread** (...): Expands arrays/objects into individual elements
- **Rest** (...): Collects multiple elements into an array

Hotel Booking Examples - Step by Step:

```

// SPREAD OPERATOR: Combining hotel data
const baseHotel = { name: "Grand Palace", location: "Foshan" };
const hotelDetails = { rating: 4.5, rooms: 100 };
const fullHotelInfo = { ...baseHotel, ...hotelDetails };
console.log(fullHotelInfo);
// Output: { name: "Grand Palace", location: "Foshan", rating: 4.5, rooms: 100 }

// Spread for adding rooms to existing list
const existingRooms = ["101", "102", "103"];
const newRooms = ["201", "202"];
const allRooms = [...existingRooms, ...newRooms];
console.log(allRooms); // ["101", "102", "103", "201", "202"]

// Spread for copying objects (avoiding reference issues)
const originalHotel = { name: "Grand Palace", price: 200 };
const updatedHotel = { ...originalHotel, price: 250 };
console.log(originalHotel.price); // 200 (unchanged)
console.log(updatedHotel.price); // 250 (updated)

// REST PARAMETER: Flexible guest counting
function calculateTotalGuests(...guestCounts) {
  return guestCounts.reduce((total, count) => total + count, 0);
}

const total = calculateTotalGuests(2, 3, 1, 4); // 10 guests
console.log(`Total guests across all bookings: ${total}`);

// Rest with other parameters
function processBooking(guestName, ...roomNumbers) {
  console.log(`Guest: ${guestName}`);
  console.log(`Rooms: ${roomNumbers.join(', ')}`);
}

processBooking("Leo", "101", "102", "103");
// Output:
// Guest: Leo
// Rooms: 101, 102, 103

```

Your Turn :

```

// TODO: Practice spread and rest operators
// 1. Use spread to combine two hotel amenity arrays
// 2. Use spread to create a copy of a hotel object and update the price
// 3. Create a function with rest parameter to calculate total capacity
// 4. Use spread to add new guests to an existing guest list

const hotelAmenities1 = ["WiFi", "Pool"];
const hotelAmenities2 = ["Spa", "Gym", "Restaurant"];

const hotel = { name: "Grand Palace", price: 200, rating: 4.5 };
const guests = ["John", "Jane"];

```

```
// Your code here:
```

Use spread for immutable updates (creating new objects/arrays instead of modifying existing ones). This is crucial for React state management.

Practice: Use spread/rest operators to:

1. Combine hotel amenities from multiple sources
2. Calculate total capacity across multiple rooms
3. Add new guests to existing guest list

Activity 6: Array Methods for Hotel Management

Array Methods for Hotel Data

Understanding Array Methods: Array methods like `map`, `filter`, `reduce`, `find`, and `sort` are powerful tools for transforming and manipulating hotel data without using traditional loops.

Hotel Booking Examples - Step by Step:

```
// Sample hotel data
const hotels = [
  { id: 1, name: "Grand Palace", price: 200, rating: 4.5, available: true },
  { id: 2, name: "Seaside Resort", price: 150, rating: 4.0, available: true },
  { id: 3, name: "Mountain Lodge", price: 180, rating: 4.8, available: false },
  { id: 4, name: "City Center Inn", price: 120, rating: 3.5, available: true }
];

// FILTER: Get only available hotels
const availableHotels = hotels.filter(hotel => hotel.available);
console.log("Available hotels:", availableHotels);

// MAP: Create hotel names list
const hotelNames = hotels.map(hotel => hotel.name);
console.log("Hotel names:", hotelNames);
```

```

// MAP with transformation: Create formatted hotel descriptions
const hotelDescriptions = hotels.map(hotel =>
  `${hotel.name} - ${hotel.price}/night - ${hotel.rating} stars`
);
console.log(hotelDescriptions);

// REDUCE: Calculate average price
const averagePrice = hotels.reduce((acc, hotel) => acc + hotel.price, 0) /
hotels.length;
console.log(`Average price: ${averagePrice}`);

// FIND: Get first hotel under $150
const affordableHotel = hotels.find(hotel => hotel.price < 150);
console.log("Affordable hotel:", affordableHotel);

// SOME: Check if any hotel has rating >= 4.5
const hasHighRatedHotel = hotels.some(hotel => hotel.rating >= 4.5);
console.log("Has high-rated hotel:", hasHighRatedHotel);

// EVERY: Check if all hotels are available
const allAvailable = hotels.every(hotel => hotel.available);
console.log("All available:", allAvailable);

// SORT: Sort hotels by price (low to high)
const sortedByPrice = [...hotels].sort((a, b) => a.price - b.price);
console.log("Sorted by price:", sortedByPrice);

// COMBINED: Get names of available hotels under $200
const affordableAvailableHotels = hotels
  .filter(hotel => hotel.available && hotel.price < 200)
  .map(hotel => hotel.name);
console.log("Affordable available hotels:", affordableAvailableHotels);

```

Your Turn:

```

// TODO: Practice array methods
// 1. Filter hotels with rating 4.0 or higher
// 2. Map hotels to create "Name - $Price" format
// 3. Find the most expensive hotel using reduce
// 4. Sort hotels by rating (high to low)
// 5. Check if any hotel is not available

const hotels = [
  { id: 1, name: "Grand Palace", price: 200, rating: 4.5, available: true },
  { id: 2, name: "Seaside Resort", price: 150, rating: 4.0, available: true },
  { id: 3, name: "Mountain Lodge", price: 180, rating: 4.8, available: false },
  { id: 4, name: "City Center Inn", price: 120, rating: 3.5, available: true }
];

// Your code here:

```

```
// Your outputs:
```

Array methods are chainable and return new arrays, making them perfect for functional programming and React data transformations.

Practice:

1. Filter hotels with rating 4.0 or higher
2. Map hotels to create "Name - \$Price" format
3. Find the most expensive hotel using reduce
4. Sort hotels by price (low to high)

Activity 7: Looping Over Hotel Data

Looping Over Hotel Data

Understanding Modern Loops: Modern JavaScript provides elegant ways to iterate over data structures without traditional `for` loops.

Hotel Booking Examples:

```
// for...of loop (best for arrays)
const hotels = [
  { id: 1, name: "Grand Palace", rating: 4.5 },
  { id: 2, name: "Seaside Resort", rating: 4.0 },
  { id: 3, name: "Mountain Lodge", rating: 4.8 }
];

console.log("=== for...of Loop ===");
for (const hotel of hotels) {
  console.log(`${hotel.name} - ${hotel.rating} stars`);
}

// for...in loop (best for objects)
const hotelDetails = {
  name: "Grand Palace",
  location: "New York",
  rating: 4.5,
  price: 200
};

console.log("=== for...in Loop ===");
for (const key in hotelDetails) {
```

```

    console.log(` ${key}: ${hotelDetails[key]} `);
}

// Object.entries() for key-value pairs
console.log("=== Object.entries() ===");
for (const [key, value] of Object.entries(hotelDetails)) {
    console.log(` ${key}: ${value} `);
}

// Object.keys() for just keys
console.log("=== Object.keys() ===");
for (const key of Object.keys(hotelDetails)) {
    console.log(`Key: ${key} `);
}

// Object.values() for just values
console.log("=== Object.values() ===");
for (const value of Object.values(hotelDetails)) {
    console.log(`Value: ${value} `);
}

// Map iteration
const hotelAmenities = new Map([
    ["WiFi", true],
    ["Pool", true],
    ["Spa", false],
    ["Gym", true]
]);

console.log("=== Map Iteration ===");
for (const [amenity, available] of hotelAmenities) {
    console.log(` ${amenity}: ${available ? "Available" : "Not Available"} `);
}

```

Your Turn:

```

// TODO: Practice modern loops
// 1. Loop through hotels and display formatted information
// 2. Loop through guest bookings and calculate total revenue
// 3. Loop through amenities and count available ones
// 4. Use Object.entries() to create a summary string

const bookings = [
    { guest: "John", room: "101", price: 200 },
    { guest: "Jane", room: "102", price: 250 },
    { guest: "Bob", room: "103", price: 180 }
];

// Your code here:

// Your outputs:

```

Use `for...of` for arrays and `Object.entries()` for objects when you need both keys and values. Avoid `for...in` for arrays as it can include inherited properties.

Practice: Loop through guest bookings and display:

- Guest name
- Room number
- Check-in date
- Total price

Activity 8: Classes for Hotel Data Modeling

Classes for Hotel Data

Understanding Classes: Classes provide a blueprint for creating objects with shared properties and methods. They're essential for organizing hotel data and business logic.

Hotel Booking Examples

```
// Hotel class
class Hotel {
  constructor(name, location, rating, basePrice) {
    this.name = name;
    this.location = location;
    this.rating = rating;
    this.basePrice = basePrice;
    this.rooms = [];
  }

  // Method to add room
  addRoom(roomNumber, type, price) {
    this.rooms.push({ roomNumber, type, price });
  }

  // Method to calculate total revenue
  calculateRevenue() {
    return this.rooms.reduce((total, room) => total + room.price, 0);
  }

  // Method to get hotel summary
  getSummary() {
```



```

        return `${this.name} in ${this.location} - ${this.rating} stars -
        ${this.basePrice}/night`;
    }

    // Static method (called on class, not instance)
    static compareByRating(a, b) {
        return b.rating - a.rating;
    }
}

// Using the Hotel class
const hotel = new Hotel("Grand Palace", "Foshan", 4.5, 200);
hotel.addRoom("101", "Single", 150);
hotel.addRoom("102", "Double", 200);
hotel.addRoom("103", "Suite", 300);

console.log(hotel.getSummary());
console.log("Total Revenue:", hotel.calculateRevenue());

// Room class with inheritance
class Room {
    constructor(roomNumber, type, price, capacity) {
        this.roomNumber = roomNumber;
        this.type = type;
        this.price = price;
        this.capacity = capacity;
        this.available = true;
    }

    book() {
        if (this.available) {
            this.available = false;
            return true;
        }
        return false;
    }

    release() {
        this.available = true;
    }
}

// LuxuryRoom extends Room
class LuxuryRoom extends Room {
    constructor(roomNumber, price, capacity, amenities) {
        super(roomNumber, "Luxury", price, capacity);
        this.amenities = amenities;
    }

    getDetails() {
        return `${super.constructor.name} ${this.roomNumber} - ${this.price} -
        Amenities: ${this.amenities.join(", ")}`;
    }
}

const luxuryRoom = new LuxuryRoom("Penthouse", 500, 4, ["Jacuzzi",

```

```
"Balcony", "Mini Bar"]]);  
console.log(luxuryRoom.getDetails());
```

Your Turn:

```
// TODO: Practice classes  
// 1. Create a Guest class with name, email, and booking history  
// 2. Add methods to add bookings and get total spent  
// 3. Create a Booking class that links Guest and Room  
// 4. Use inheritance to create VIPGuest with special benefits  
  
// Your code here:
```

Classes are perfect for modeling real-world entities like hotels, rooms, and guests. They help organize code and make it more maintainable.

Activity 9: Modern JavaScript Features

Optional Chaining & Nullish Coalescing

Understanding Modern Features: Modern JavaScript provides operators to handle null/undefined values gracefully.

Hotel Booking Examples:

```
// Optional chaining (?.)  
const hotel = {  
  name: "Grand Palace",  
  location: "Foshan",  
  contact: {  
    phone: "+1234567890"  
  }  
};  
  
// OLD WAY: Risky nested property access  
const phone = hotel.contact && hotel.contact.phone;  
  
// NEW WAY: Safe with optional chaining  
const safePhone = hotel.contact?.phone; // Returns undefined if contact  
doesn't exist  
const email = hotel.contact?.email; // Returns undefined (not error)  
  
console.log("Phone:", safePhone);
```

```

console.log("Email:", email); // undefined (no error)

// Deep optional chaining
const booking = {
  hotel: {
    name: "Grand Palace",
    address: {
      city: "Foshan",
      zip: "10001"
    }
  }
};

const city = booking?.hotel?.address?.city; // "Foshan"
const country = booking?.hotel?.address?.country; // undefined

// Nullish coalescing (??)
const hotelRating = hotel.rating ?? 4.0; // Use 4.0 if rating is
null/undefined
const hotelName = hotel.name ?? "Unknown Hotel";

console.log("Rating:", hotelRating);
console.log("Name:", hotelName);

// Difference between || and ??
const zero = 0;
const result1 = zero || 10; // 10 (0 is falsy)
const result2 = zero ?? 10; // 0 (0 is not null/undefined)

console.log("|| operator:", result1);
console.log("?? operator:", result2);

// Combining optional chaining and nullish coalescing
const guestEmail = booking?.guest?.email ?? "No email provided";
console.log("Guest email:", guestEmail);

```

Your Turn:

```

// TODO: Practice modern JavaScript features
// 1. Use optional chaining to safely access nested hotel data
// 2. Use nullish coalescing to provide default values
// 3. Combine both for robust data access
// 4. Handle missing data gracefully

const hotelData = {
  name: "Grand Palace",
  amenities: ["WiFi", "Pool"]
  // Note: contact info might be missing
};

// Your code here:

```

Always use optional chaining when accessing nested properties that might not exist.
Use nullish coalescing for default values when you want to distinguish between null/undefined and other falsy values.

Activity 10: Promises for Asynchronous Operations

Promises for Hotel API Calls

Understanding Promises: Promises represent the eventual completion (or failure) of an asynchronous operation, like fetching hotel data from an API.

Hotel Booking Examples - Step by Step:

```
// Creating a promise
function fetchHotelData(hotelId) {
  return new Promise((resolve, reject) => {
    // Simulate API call
    setTimeout(() => {
      if (hotelId > 0) {
        resolve({
          id: hotelId,
          name: "Grand Palace Hotel",
          location: "Foshan",
          rating: 4.5
        });
      } else {
        reject(new Error("Invalid hotel ID"));
      }
    }, 1000);
  });
}

// Using the promise
console.log("Fetching hotel data...");

fetchHotelData(1)
  .then(hotel => {
    console.log("Hotel found:", hotel.name);
    return hotel;
  })
  .then(hotel => {
    console.log(`Rating: ${hotel.rating} stars`);
  })
  .catch(error => {
    console.error("Error:", error.message);
  });
```

```

    })
    .finally(() => {
      console.log("Fetch operation complete");
    });

// Async/await (modern way to handle promises)
async function getHotelDetails(hotelId) {
  try {
    console.log("Fetching hotel details...");
    const hotel = await fetchHotelData(hotelId);
    console.log(`Hotel: ${hotel.name}`);
    console.log(`Location: ${hotel.location}`);
    return hotel;
  } catch (error) {
    console.error("Failed to fetch hotel:", error.message);
    return null;
  }
}

getHotelDetails(1);

// Promise.all for multiple requests
function fetchRoomPrices(roomIds) {
  const promises = roomIds.map(id =>
    new Promise(resolve => {
      setTimeout(() => resolve({ id, price: Math.random() * 300 + 100 }),
500);
    })
  );

  return Promise.all(promises);
}

fetchRoomPrices([1, 2, 3])
  .then(prices => {
    console.log("Room prices:", prices);
  });

```

Your Turn:

```

// TODO: Practice promises and async/await
// 1. Create a promise to simulate booking a room
// 2. Use async/await to handle the promise
// 3. Handle errors with try/catch
// 4. Use Promise.all to fetch multiple hotel data

// Your code here:

// Your output here:

```

Always use `async/await` instead of `.then()` chains for better readability. Always handle errors with `try/catch` when using `async/await`.

Main Class Activity: Hotel Data Management System

Task: Build a hotel data management system using JavaScript ES6+ features

Requirements:

- Create hotel data structure with proper types
- Implement functions for hotel operations
- Use array methods for filtering and searching
- Apply destructuring and spread operators
- Create hotel booking calculations
- Use classes for hotel data modeling
- Implement modern JavaScript features

Steps:

1. Create Hotel and Room classes
2. Implement search function by location
3. Implement filter function by price range
4. Implement sort function by rating
5. Calculate total revenue for all hotels
6. Find hotel with most amenities
7. Create booking summary function
8. Use optional chaining for safe data access
9. Implement async hotel data fetching

Homework: Complete Hotel Data System

Task 1: Enhanced Hotel Functions

- Add discount calculation function
- Implement room availability checking

- Create guest validation function
- Add booking conflict detection

Task 2: Hotel Data Operations

- Implement hotel CRUD operations
- Add hotel search with multiple criteria
- Create hotel comparison function
- Implement hotel recommendation system

Task 3: Preparation for Next Session

Read before next class:

- TypeScript Basics: <https://www.typescriptlang.org/docs/handbook/intro.html>
 - TypeScript for JavaScript Programmers:
<https://www.typescriptlang.org/docs/handbook/types-from-js.html>
-

Summary: JavaScript ES6+ for Hotel Data

Key Takeaways:

- **Modern JavaScript features** make hotel data handling more efficient
- **Array methods** simplify hotel data manipulation
- **Template literals** improve hotel messaging
- **Destructuring** makes hotel data extraction cleaner
- **Spread/rest operators** enable flexible hotel operations

Next Session: We'll add TypeScript types to make our hotel data more robust and maintainable.